

Neural Networks and Deep Learning

DL Specialization, Course 1 — detailed notes (Weeks 1–4)

Logistic regression as a NN · vectorization · shallow & deep networks

1. Week 1 — Introduction to Deep Learning

1.1 What is a neural network

A single neuron = linear regression followed by an activation. It computes $z = \mathbf{w}^\top \mathbf{x} + b$, then applies a non-linearity $a = g(z)$. A **perceptron** uses a hard threshold (outputs 0/1); replacing that step with a smooth **sigmoid** lets small weight changes produce small output changes — which is what makes gradient-based learning possible. A sigmoid neuron is exactly logistic regression.

Stack neurons into layers: hidden layers automatically discover the features connecting inputs to outputs — that's what deep learning is good at. **ReLU** ($\max(0, z)$) is the standard hidden activation today; it trains much faster than sigmoid.

1.2 Supervised learning with neural networks

Almost all economic value from deep learning today is supervised learning (learn a map $X \rightarrow Y$). Architectures by data type:

Standard NN	structured / tabular data (databases)	
CNN	images (computer vision)	Structured data
RNN	sequences: speech, text, time series	

= tables with defined features; **unstructured** = images, audio, text.

1.3 Why is deep learning taking off?

Three forces converged:

1. **Data** — digitization (mobile, IoT) produced huge labeled datasets. Large NNs keep improving with more data, while traditional algorithms (SVM, logistic regression) plateau.
2. **Computation** — GPUs, faster CPUs, ASICs.
3. **Algorithms** — e.g. switching sigmoid $\rightarrow$ ReLU fixed the vanishing-gradient slowdown, making training much faster.

Faster training $\Rightarrow$ faster iteration on the idea $\rightarrow$ code $\rightarrow$ experiment loop $\Rightarrow$ faster progress.

2. Week 2 — Neural Network Basics (logistic regression)

2.1 Notation

m	number of training examples
n_x	size of input vector $\mathbf{x}$
$X \in \mathbb{R}^{n_x \times m}$	inputs stacked as <i>columns</i> (each example a column)
$Y \in \mathbb{R}^{1 \times m}$	labels $[y^{(1)} \dots y^{(m)}]$
$\mathbf{w} \in \mathbb{R}^{n_x}, b \in \mathbb{R}$	parameters
$\hat{y} = a$	prediction = $P(y=1 \mid \mathbf{x})$

Stacking examples as *columns* (shape $n_x \times m$) is the course convention and makes the vectorized math below line up. Different from many textbooks that use rows.

2.2 Logistic regression

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Andrew keeps b separate (rather than folding it in as w_0).

2.3 Cost function

Squared error is non-convex here, so use log loss. For one example:

$$L(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y})).$$

Intuition: if $y=1$, $L = -\log \hat{y}$ pushes $\hat{y} \rightarrow 1$; if $y=0$, $L = -\log(1 - \hat{y})$ pushes $\hat{y} \rightarrow 0$. The cost is the average over all examples:

$$J(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}).$$

L = one example; J = whole training set. J is convex.

2.4 Gradient descent

$$\mathbf{w} := \mathbf{w} - \alpha \frac{\partial J}{\partial \mathbf{w}}, \quad b := b - \alpha \frac{\partial J}{\partial b}.$$

α = learning rate; the derivative gives the downhill direction. In logistic regression people initialize $\mathbf{w}, b$ to 0 (random not needed — that's only for real NNs, see Week 3).

2.5 Derivatives via a computation graph

Deep learning frameworks compute gradients by the **chain rule** on a left-to-right computation graph, then propagate derivatives *right to left*. Convention: $dvar = \partial J / \partial var$.

For one example with features x_1, x_2 :

$$\begin{aligned} z &= w_1 x_1 + w_2 x_2 + b, & a &= \sigma(z), & L &= -(y \log a + (1 - y) \log(1 - a)) \\ dz &= a - y, & dw_1 &= x_1 dz, & dw_2 &= x_2 dz, & db &= dz. \end{aligned}$$

The clean result $dz = a - y$ falls out of $\frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z}$.

2.6 Gradient descent on m examples (the loop version)

```
J=0; dw1=0; dw2=0; db=0
for i in range(m):
    z = w1*x1[i] + w2*x2[i] + b
    a = sigmoid(z)
    J += -(y[i]*log(a) + (1-y[i])*log(1-a))
    dz = a - y[i]
    dw1 += x1[i]*dz; dw2 += x2[i]*dz; db += dz
J/=m; dw1/=m; dw2/=m; db/=m
w1 -= alpha*dw1; w2 -= alpha*dw2; b -= alpha*db
```

Two nested loops (over examples, over features) — far too slow for deep learning. **Vectorization** removes both.

2.7 Vectorization

`np.dot` runs in parallel (SIMD on CPU/GPU) — vastly faster than Python loops. Rule: *avoid explicit for-loops whenever possible*.

Forward pass over the whole training set at once:

$$Z = \mathbf{w}^\top X + b,$$

$$A = \sigma(Z),$$

```
Z = np.dot(w.T, X) + b    # b broadcasts; Z is (1, m)
A = 1 / (1 + np.exp(-Z)) # A is (1, m)
```

Vectorized gradients:

$$dZ = A - Y, \quad d\mathbf{w} = \frac{1}{m} X dZ^\top, \quad db = \frac{1}{m} \text{np.sum}(dZ).$$

One gradient-descent step, no loops:

```
Z = np.dot(w.T, X) + b
A = sigmoid(Z)
dZ = A - Y
dw = np.dot(X, dZ.T) / m
db = np.sum(dZ) / m
w -= alpha*dw; b -= alpha*db
```

(You still loop over gradient-descent *iterations* — that loop is unavoidable.)

2.8 Broadcasting & NumPy gotchas

- **Broadcasting:** an (m, n) matrix combined with a $(1, n)$ row copies the row m times (and a $(m, 1)$ column n times), then does the op element-wise.
- `obj.sum(axis=0)` sums columns; `axis=1` sums rows.
- `reshape` is $O(1)$ — use it liberally to force shapes.

Rank-1 arrays are a bug source. `np.random.randn(5)` has shape $(5,)$ — its transpose does nothing. Always make vectors explicit column shape $(5, 1)$, and sprinkle `assert(a.shape==(5,1))`. Fix any rank-1 array with `reshape`.

2.9 Sigmoid derivative & image trick

```
s = sigmoid(x); ds = s*(1-s)    # dsigma/dz
v = image.reshape(H*W*3, 1)     # flatten an image to a
    vector
```

Normalizing inputs makes gradient descent converge faster.

2.10 Steps to build a NN (the recurring recipe)

1. Define the model structure (# inputs, # outputs).
2. Initialize the parameters.
3. Loop: forward prop (compute loss) → backward prop (compute gradients) → update parameters.

Preprocess/normalize the data; tuning α (a **hyperparameter**) matters a lot.

3. Week 3 — Shallow Neural Networks (one hidden layer)

3.1 From logistic unit to a layer

Logistic regression: $\mathbf{x} \rightarrow z \rightarrow a \rightarrow L$. A 2-layer NN just repeats this: layer 1 produces $\mathbf{a}^{[1]}$, which feeds layer 2. A NN is a stack of logistic-regression-like units.

3.2 Notation & representation

Superscript $[l]$ = layer; the input layer is $\mathbf{a}^{[0]} = \mathbf{x}$, hidden layer output $\mathbf{a}^{[1]}$, output $\mathbf{a}^{[2]} = \hat{y}$. A “2-layer NN” has 1 hidden + 1 output layer (*input layer isn't counted*). The hidden layer is “hidden” because its values aren't in the training data.

3.3 Computing the output

For one example, with $n^{[1]}$ hidden units and n_x inputs:

$$\begin{aligned} \mathbf{z}^{[1]} &= W^{[1]} \mathbf{x} + \mathbf{b}^{[1]}, & \mathbf{a}^{[1]} &= g^{[1]}(\mathbf{z}^{[1]}), \\ \mathbf{z}^{[2]} &= W^{[2]} \mathbf{a}^{[1]} + \mathbf{b}^{[2]}, & \mathbf{a}^{[2]} &= \sigma(\mathbf{z}^{[2]}). \end{aligned}$$

Shapes (with $n^{[1]}$ hidden units): $W^{[1]}$ is $(n^{[1]}, n_x)$, $\mathbf{b}^{[1]}$ is $(n^{[1]}, 1)$; $W^{[2]}$ is $(1, n^{[1]})$, $\mathbf{b}^{[2]}$ is $(1, 1)$.

3.4 Vectorizing across m examples

Stack examples as columns ($X = A^{[0]}$, shape $n_x \times m$):

$$\begin{aligned} Z^{[1]} &= W^{[1]} A^{[0]} + \mathbf{b}^{[1]}, & A^{[1]} &= g^{[1]}(Z^{[1]}), \\ Z^{[2]} &= W^{[2]} A^{[1]} + \mathbf{b}^{[2]}, & A^{[2]} &= \sigma(Z^{[2]}). \end{aligned}$$

Columns index examples; rows index units. $Z^{[1]}, A^{[1]}$ are $(n^{[1]}, m)$.

3.5 Activation functions

Name	Formula	Range / use
Sigmoid	$\frac{1}{1+e^{-z}}$	$(0, 1)$; only for the <i>output</i> in binary classification
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$; better than sigmoid for hidden units (zero-centered)
ReLU	$\max(0, z)$	default hidden activation; no saturation for $z > 0$
Leaky ReLU	$\max(0.01z, z)$	small negative slope; rarely needed over ReLU

- Tanh centres activations near 0, helping the next layer learn — generally beats sigmoid in hidden layers.
- Sigmoid/tanh *saturate* for large $|z|$ (slope $\rightarrow 0$), slowing gradient descent. ReLU avoids this and is the safe default.

Why non-linear activations are necessary

Remove the activation (use a linear one) and every layer composes into a single linear function — the whole network collapses to plain logistic/linear regression, no matter how deep. A linear activation is only ever used at the *output* of a regression problem.

3.6 Derivatives of the activations

$$\begin{aligned}\sigma'(z) &= \sigma(z)(1 - \sigma(z)), & \tanh'(z) &= 1 - \tanh^2(z), \\ \text{ReLU}'(z) &= \begin{cases} 0 & z < 0 \\ 1 & z \geq 0 \end{cases}, & \text{LeakyReLU}'(z) &= \begin{cases} 0.01 & z < 0 \\ 1 & z \geq 0 \end{cases}.\end{aligned}$$

3.7 Gradient descent — full backprop (2-layer)

Forward: $Z^{[1]} = W^{[1]}X + b^{[1]}$, $A^{[1]} = g^{[1]}(Z^{[1]})$, $Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$, $A^{[2]} = \sigma(Z^{[2]})$.

$$\begin{aligned}dZ^{[2]} &= A^{[2]} - Y, & dW^{[2]} &= \frac{1}{m} dZ^{[2]} A^{[1]\top}, & db^{[2]} &= \frac{1}{m} \text{np.sum}(dZ^{[2]}, \text{axis}=1), \\ dZ^{[1]} &= (W^{[2]\top} dZ^{[2]}) * g'^{[1]}(Z^{[1]}), & dW^{[1]} &= \frac{1}{m} dZ^{[1]} X^\top, & db^{[1]} &= \frac{1}{m} \text{np.sum}(dZ^{[1]}, \text{axis}=1).\end{aligned}$$

The $*$ in $dZ^{[1]}$ is *element-wise*. Transposes appear to keep matrix dimensions consistent. Use `keepdims=True` in the sums.

3.8 Random initialization

Never initialize W to zeros in a NN. All hidden units would compute the same function (symmetry) and update identically forever — the hidden layer learns nothing. (Initializing $b = 0$ is fine.)

```
W1 = np.random.randn(n1, n0) * 0.01    # small random -> breaks symmetry
b1 = np.zeros((n1, 1))                  # zero bias is OK
```

Multiply by a *small* constant (0.01): large weights push sigmoid/tanh into saturation from the start, slowing learning. (For deep nets, 0.01 is refined — see He init in Course 2.)

4. Week 4 — Deep Neural Networks

4.1 Notation for an L -layer network

Shallow = 1–2 layers; **deep** = 3+. L = number of layers, $n^{[l]}$ = units in layer l (with $n^{[0]} = n_x$, $n^{[L]}$ = output size), $g^{[l]}$ = its activation, $\mathbf{a}^{[l]} = g^{[l]}(\mathbf{z}^{[l]})$, $\mathbf{x} = \mathbf{a}^{[0]}$, $\hat{y} = \mathbf{a}^{[L]}$.

4.2 Forward propagation (general layer l)

$$\begin{aligned}\text{one example: } \mathbf{z}^{[l]} &= W^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}, & \mathbf{a}^{[l]} &= g^{[l]}(\mathbf{z}^{[l]}), \\ \text{whole batch: } Z^{[l]} &= W^{[l]} A^{[l-1]} + \mathbf{b}^{[l]}, & A^{[l]} &= g^{[l]}(Z^{[l]}).\end{aligned}$$

A **for-loop over layers** $l = 1 \dots L$ is fine (and necessary) here — you can't vectorize across layers.

4.3 Getting matrix dimensions right

$$W^{[l]} : (n^{[l]}, n^{[l-1]}), \quad \mathbf{b}^{[l]} : (n^{[l]}, 1), \quad Z^{[l]}, A^{[l]}, dZ^{[l]}, dA^{[l]} : (n^{[l]}, m).$$

$dW^{[l]}$ has the same shape as $W^{[l]}$; $db^{[l]}$ same as $\mathbf{b}^{[l]}$. Debug shapes with pencil and paper.

4.4 Why deep representations?

Layers build features from simple to complex — e.g. images: *edges* $\rightarrow$ *face parts* $\rightarrow$ *faces*; audio: *low-level sound* $\rightarrow$ *phonemes* $\rightarrow$ *words*. **Circuit-theory intuition:** some functions a small deep network can compute would need an *exponentially larger* shallow (single-hidden-layer) network. Still, start simple (logistic $\rightarrow$ shallow) and deepen only as needed.

4.5 Building blocks: forward & backward for a layer

Each layer l is a forward function (input $A^{[l-1]}$, output $A^{[l]}$, *cache* $Z^{[l]}$) and a backward function (input $dA^{[l]}$, output $dA^{[l-1]}, dW^{[l]}, db^{[l]}$).

Forward (layer l)

```
Z[l] = W[l] @ A[l-1] + b[l]
A[l] = g[l](Z[l]) # cache Z[l] for backprop
```

Backward (layer l)

$$\begin{aligned} dZ^{[l]} &= dA^{[l]} * g'^{[l]}(Z^{[l]}), & dW^{[l]} &= \frac{1}{m} dZ^{[l]} A^{[l-1]\top}, \\ db^{[l]} &= \frac{1}{m} \text{np.sum}(dZ^{[l]}, \text{axis}=1, \text{keepdims}=\text{True}), & dA^{[l-1]} &= W^{[l]\top} dZ^{[l]}. \end{aligned}$$

The chain starts from the output layer with

$$dA^{[L]} = -\frac{y}{a} + \frac{1-y}{1-a} \quad (\text{derivative of log loss}).$$

Then update every layer: $W^{[l]} := W^{[l]} - \alpha dW^{[l]}$, $b^{[l]} := b^{[l]} - \alpha db^{[l]}$.

4.6 Parameters vs. hyperparameters

Parameters (learned): $W^{[l]}, b^{[l]}$.

Hyperparameters (you choose, they *control* the parameters): learning rate α , # iterations, # hidden layers L , # units $n^{[l]}$, choice of activation. (Later: momentum, mini-batch size, regularization.)

Deep learning is empirical — you try hyperparameter values, observe J , and iterate.

4.7 Relation to the brain

The “NN = brain” analogy is loose. A logistic unit vaguely resembles a neuron, but no one fully understands real neurons. Deep learning’s real strength is flexibly learning complex $X \rightarrow Y$ mappings from data — not biological fidelity.

The through-line of Course 1: everything is the same three-step recipe — *initialize* $\rightarrow$ (*forward: compute cost*) $\rightarrow$ (*backward: compute gradients*) $\rightarrow$ *update*. A deep network is just logistic regression’s forward/backward equations wrapped in a loop over layers, with the caches passed along and $dZ = A - Y$ at the output seeding the backward chain.